Amortizirana časovna zahtevnost

Operacija ima amort. čas zahtevnost c, če kakršno koli zaporedje n operacij zahteva največjemu $c \cdot n$ enot časa

Primer: Dvojiški števec predstavljen s tabelo ničel in enic B, ki predstavlja število $\sum B[i] \cdot 2^i$

Za posamezno povečanje $O(\log n)$ časa za n korakov potrebujemo $O(n \log(n))$ za en korak je amort. čas. zahtevnost $O(\log(n))$

Agregacijska metoda

$$C = \frac{\sum_{i=n}^{n} c_i}{n}$$

1. Dvojiški števec

$$\sum c_i = n + \lfloor \tfrac{n}{2} \rfloor + \ldots + \lfloor \tfrac{n}{2^{\log n}} \rfloor = \leq \sum_{k=0}^{\infty} \frac{n}{2^k} = 2n$$

$\Rightarrow$ $C = 2$ Amortizirana čas. zah. je $O(1)$

2 Dinamična tabela

Pri vsakem koraku vstavimo 1 element

Na koraku $2^k + 1$ prestavimo te 2^k el.

$$\sum c_i = n + 1 + 2 + 4 + 8 + \ldots + 2^{\lfloor \log n \rfloor} \leq$$

$$\leq n + \sum_{i=0}^{\lfloor \log n \rfloor} 2^i = n + 2^{\lfloor \log n \rfloor + 1} - 1$$

$$\leq n + 2^{\log(n)+1} = 3n$$

Računska metoda

Ideja: Operacije lahke „plačajo" del stroška kasnejših operacij

1. Dvojiški števec
   - Vsaka sprememba iz 0 na 1 plača 2 enoti. Eno zase, eno za morebitno spremembo 1→0
   - Sprememba 1→0 pokriti privarčevana sredstva

   Operacija bo plačala $\hat{c}_i$ tako, da bo $\sum c_i \leq \sum \hat{c}_i$

Na vsakem koraku plačamo natanko 2 enoti.

Hkrati je ta dovolj, saj vsaka sprememba 1→0 sled. spremembi 0→1

3. Unija z rangom

   Glej PSA1

   Dokažimo da je amortizirana čas. zah. $O(\log(n))$

$I_0 = [1, 1]$ $I_2 = [2, 2]$ $I_3 = [3, 4]$ $I_3 = 5, 16]$

$$I_k = [2^l + 1, 2^{2^l}] = \{x \in \mathbb{N} \mid \log^* x = k\}$$

$\forall$ 26.2 manyhola

5.3

Predvorba statičnih podatkovnih struktur v dinamične

dinamična .... dodajamo in vzemamo elemente
poldinamična .... samo eno ali drugo
statična .... samo gledamo elemente

Elemente lahko shranimo v statično tabelo.
Ko odstranimo element, označimo v neki drugi tabeli vrednost False, da ga ne moremo več vrniti

Recimo, da imamo P.S. nad elementi iz množice X s stopnjo $S(X)$, ki podpira
- predpripravo $\iota : P(X) \to S(X)$
- poizvedbe $\varphi : S(X) \times Q \to A$

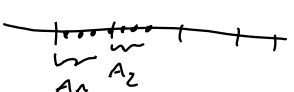
tako da poizvedbe lahko razpadejo preko operacije $\oplus : \mathcal{R} \times \mathcal{R} \to \mathcal{R}$

da je

$$\varphi(\bigcup(A_1 \cup A_2), g) = \varphi(\iota(A_1), g) \oplus \varphi(\iota(A_2), g)$$

Primeri:
- iskanje elementa v tabeli
  - ι .... uredi tabelo
  - φ .... poišče z bisekcijo
  - $\oplus$ .... $\vee$
- iskanje najbližje točke v ravnini v $A^{kom} \subseteq \mathbb{R}^k$
  - ι .... k-d drevo
  - φ ..... najbližja točka
  - $\oplus$ ..... izbere izmed dveh točk bližnjo

Protiprimer
Število enotnih intervalov s krajišči v A



ne razpade

Ideja je, da vodimo svoje $L_0, L_1, \dots, L_k$

v L_i je 2^i elementov, ki so urejeni.

Nov element dodajamo na najnižji nivo

če je zasedeno, jih potisnemo skupaj in demo gor

Naj bo • časovna zahtevnost $L(A)$ enaka $P(n)$ $n = |A|$

• čas. zaht. $p(L(A), g)$ je $Q(n)$

• Naj bo P superaditivna $(P(m+n) \geq P(m) + P(n))$

Tedaj je amortizirana čas. zaht. vstavljanja enaka

$$I'(n) = \begin{cases} O(P(n)) & P(n) \in \Theta(n^{1+\varepsilon}) \quad \varepsilon > 0 \\ P(n)\, O(\log n); & \text{v vseh primerih} \end{cases}$$

' označuje zahtevnosti na novi strukturi

Časovna zahtevnost poizvedb:

$$Q'(n) = \begin{cases} O(Q(n)) & ; \quad Q(n) \in \Theta(n^{\varepsilon}) \quad \varepsilon > 0 \\ Q(n)\, O(\log n) & ; \quad \text{v vseh primerih} \end{cases}$$

Dokaz:

1) • $Q'(n) = \Theta(Q(n)\log(n))$ DN

2) • $Q'(n) = \Theta(n^{\varepsilon})$ z $\varepsilon > 0 \Rightarrow Q'(n) = \Theta(Q(n))$ DN

3) • $P'(n)$ amort $= \Theta\left(\frac{P(n)\log n}{n}\right)$

4) • $P(n) = \Theta(n^{1+\varepsilon})$ za $\varepsilon > 0 \Rightarrow$

3) Recimo da smo ustavili n elementov

.... Parabom $\frac{n}{2}P(1) + \frac{n}{4}P(2) + \frac{n}{8}P(4) + \dots + P(\frac{n}{2})$

$= *$

Zaradi superaditivnosti

$P(2 \cdot \frac{n}{2}) \geq 2 \cdot P(\frac{n}{2})$

in podobno $P(m) \geq 2^{\ell} P(\frac{m}{2^{\ell}})$

$* \leq \frac{1}{2}(P(n) + P(n) + \dots + P(n)) \leq \Theta(P(n)\log(n))$

4) Ker je $P(n) = \Theta(n^{1+\varepsilon})$ je tudi $\Theta(n^{1+\varepsilon}$

zato je skupaj čas. zah.

$$\frac{1}{2}\sum_{i=0}^{\log n} \frac{n}{2^i} P(2^i) = \frac{n}{2}\sum_{i=0}^{\log n} \frac{\Theta((2^i)^{\varepsilon+1})}{\cancel{2^i}} =$$

$$\Theta\left(\frac{n}{2}\sum_{i=0}^{\log n} (2^{\varepsilon})^i\right) = \frac{n}{2}\Theta\left((2^{\varepsilon})^{\log n+1}\right) =$$

$$= \Theta(n^{\varepsilon+1}) = \Theta(P(n))$$

Hitra fourierova transformacija

Imamo različne predstavitve polinomov stopnje n z $n+1=N$ podatki

predstaviten	seštevanje	množenje	izračun vrednosti v t.	
$a_0 + \dots a_n x^n$	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n)$	
$a_n(x-x_0)\dots(x-x_1)$	???	$\mathcal{O}(n)$	$\mathcal{O}(n)$	
$(x_0,y_0)\dots(x_n,y_n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	

Poglejmo kako lahko za smiselno izbrane točke dobimo evalvacijo v času $\mathcal{O}(n \log n)$

Opazimo:

$$a_0 + a_1 x + \dots = a_0 + a_2 x^2 + a_4 x^4) + x(a_1 + a_3 x^2 + \dots)$$

oz: $p(x) = p_s(x^2) + x\, p_l(x^2)$

1) $p(1) = p_s(1) + p_l(1)$

2) $p(-1) = p_s(1) - p_l(1)$ rabima samo

3) $p_s(-1) + i\, p_l(-1)$ 2 vrednosti

4) $p_s(-1) - i\, p_l(-1)$

p_s razdelimo še naprej $p_s = p_{ss} + p_{sl}$

Naj bo $N = 2^k$ in $\omega = e^{\frac{2\pi i}{N}}$

Vidimo da velja

$$p(\omega^k) = p_s(\omega^{2k}) + \omega^k p_l(\omega^{2k})$$

$$p\left(\omega^{k+\frac{N}{2}}\right) = p(-\omega^k) = p_s(\omega^{2k}) - \omega^k p_l(\omega^{2k})$$

$$FFT([a_0, a_1, \dots a_{N-1}]) = [p(\omega^0), p(\omega^1), \dots p(\omega^{N-1})]$$

za $p(x) = a_0 + a_1 x + \dots a_{N-1} x^{N-1}$

časovna zahtevnost $O(n \log n)$

Rabimo se inverz

Za dovolj velik N (DN) velja

$$FFT(\mathrm{koef}(p \cdot q)) = FFT(\mathrm{koef}(p)) \cdot FFT(\mathrm{koef}(q))$$

Če želimo $\mathrm{koef}(pq)$ rabimo inverz

$$\begin{bmatrix} p(\omega^0) \\ \vdots \\ p(\omega^{N-1}) \end{bmatrix} = \begin{bmatrix} 1 & 1 & \cdots & 1 \\ \vdots & \omega & \cdots & \omega^{N-1} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{N-1} & \cdots & \omega^{(N-1)^2} \end{bmatrix} \begin{bmatrix} a_0 \\ \vdots \\ a_{N-1} \end{bmatrix}$$

$$[W_{ij}] = \omega^{ij}$$

W je skoraj unitarna

$$[WW^H]_{ij} = \sum_{k=0}^{N-1} \omega^{ik} \underbrace{\bar{\omega}^{kj}}_{\omega^{-kj}} = \sum_{k=0}^{N-1} (\omega^{(i-j)})^k = \begin{cases} 0 & i \neq j \\ N & i = j \end{cases}$$

$$W^{-1} = \frac{1}{N} W^H$$